

## CHATGPT ARCHIVE BROWSER

# Getting Started User Guide

A success-first guide for turning a multi-gigabyte ChatGPT export into a private, chronological, searchable archive you can actually understand.

PRESERVE

-&gt;

COLLECT

-&gt;

VERIFY

-&gt;

BROWSE

**The setup matters**

A huge ChatGPT export is not one useful document. It is a container holding conversation records, attachments, media, and account data. The safest approach is to preserve the original, create a smaller ZIP containing only the conversation JSON files, verify that working ZIP, and build the browser from it.

Version 1.1.0 | September 2026

[github.com/RandoSY/works-and-publications](https://github.com/RandoSY/works-and-publications)

MIT licensed | Local-first | No third-party Python packages required

# 1 Why do this at all?

The export preserves your history. The browser makes the history understandable.

When a ChatGPT export becomes very large, the problem is no longer "How do I save it?" The problem becomes "How do I make sense of years of conversations without opening raw JSON files or scrolling forever?"

## What OpenAI documents

OpenAI says an export may contain conversations.json, while larger exports may contain numbered conversation JSON files instead. Uploading exported conversation files can make them available as reference, but does not recreate the original chats as separate sidebar conversations.

| What the raw export is good at       | What the local browser is good at                    |
|--------------------------------------|------------------------------------------------------|
| Preserving the full account export   | Reading conversations in a human-friendly form       |
| Keeping attachments and assets       | Finding titles, phrases, projects, and periods       |
| Providing JSON records               | Putting all conversation chunks into true date order |
| Serving as a source-of-truth backup  | Moving through conversations with Previous / Next    |
| Holding everything, even bulky media | Giving you a compact catalog for later analysis      |

## The key idea

- Keep the original ChatGPT export ZIP unchanged.
- Make a second ZIP containing only the conversation JSON files.
- Build the readable browser from that smaller working ZIP.
- Use the original export whenever you need images, attachments, or other account data.

## Why not work directly from 7+ GB every time?

You can, but a clean conversation-only ZIP is easier to verify, copy, troubleshoot, reuse, and understand. It separates the conversation corpus from the bulky preservation archive.

## 2 The whole job in two stages

No command-line experience is required for the recommended Windows path.

1

### Stage 1 - prepare the conversation ZIP

Drag the ORIGINAL ChatGPT export ZIP onto prepare\_windows.bat. The helper finds every conversation JSON file, copies only those files into ChatGPT\_Conversations\_Only.zip, checks the numbered sequence, creates a manifest, and verifies the new ZIP.

**Why this matters:** You now have a small, explicit working corpus instead of a huge mixed export.

2

### Stage 2 - build the browser

Drag ChatGPT\_Conversations\_Only.zip onto run\_windows.bat. The generator creates ChatGPT\_Archive\_Browser and opens index.html.

**Why this matters:** The result is an ordinary local website that you can search and browse without changing the original export.

## What you should have when finished

```
Original ChatGPT export.zip <-- KEEP THIS
ChatGPT_Conversations_Only.zip <-- WORKING CORPUS
ChatGPT_Archive_Browser/ <-- READABLE VIEW
index.html
catalog.json
conversations/
one HTML page per conversation
```

### Three different roles

The original ZIP is the preservation artifact. The conversation-only ZIP is the working dataset. The HTML browser is the human reading interface. Keeping those roles separate is the safest mental model.

## What gets copied into the working ZIP

| Include                               | Do not include just because it is JSON |
|---------------------------------------|----------------------------------------|
| conversations.json                    | account metadata JSON                  |
| conversations-000.json                | feedback JSON                          |
| conversations-001.json, 002, 003, ... | shared-link metadata JSON              |
| All numbered conversation chunks      | unrelated export JSON files            |

## 3 Set up the tools folder

Put four small files together. Your huge export can remain where it is.

1

### Create a folder

Create a folder such as C:\ChatGPT-Archive-Tools.

**Why this matters:** Keeping the helper files together makes drag-and-drop predictable.

2

### Put these four files in it

prepare\_conversations\_zip.py, prepare\_windows.bat, chatgpt\_archive\_browser.py, and run\_windows.bat.

**Why this matters:** The BAT files are the simple Windows launchers; the PY files do the actual work.

3

### Confirm Python 3

Open Command Prompt and type `python --version`. If that fails, try `py --version`. You need Python 3.9 or newer.

**Why this matters:** The utilities use only Python standard-library modules; there is no package installation step.

```
python --version
```

```
# or
```

```
py --version
```

### If Python is not found

Install Python 3 from [python.org](https://python.org). During installation, select the option to add Python to PATH if offered. Close the old Command Prompt window, open a new one, and check the version again.

## The folder should look like this

```
C:\ChatGPT-Archive-Tools\  
prepare_conversations_zip.py  
prepare_windows.bat  
chatgpt_archive_browser.py  
run_windows.bat
```

### Do not put your personal export on GitHub

The public project contains only the software and documentation. Your original ZIP, conversation-only ZIP, and generated browser contain private conversation material and should usually remain local.

## 4 Stage 1: collect the conversation JSON files

This is the most important setup step for a huge export.

### Recommended method

Use `prepare_windows.bat`. It scans the original ZIP directly, so you do not need to unpack a 7+ GB archive just to find the conversation files.

1

## 5 Verify before you build

A first-time-user workflow should make success visible, not merely assume it.

### The preparation tool checks these things automatically

- At least one conversation JSON file was found.
- If numbered files exist, they are used as the working set; a plain conversations.json is not mixed in and duplicated.
- Numbered files are sorted numerically.
- Gaps in the number sequence produce a warning.
- Each selected JSON looks like a top-level JSON array.
- The output ZIP is reopened and tested after it is written.
- A manifest is added listing every conversation JSON file copied from the source export.

### What you should personally check

| Check                        | What good looks like                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------|
| Original export still exists | It is unchanged and remains your master backup.                                            |
| Working ZIP exists           | ChatGPT_Conversations_Only.zip is beside the source ZIP unless you chose another location. |
| Manifest exists              | CONVERSATION_FILES_MANIFEST.txt is inside the working ZIP.                                 |
| Numbered sequence            | Usually 000, 001, 002, ... with no unexpected gaps.                                        |
| Working ZIP size             | Often much smaller than the original export - this is expected.                            |

#### If you see a gap warning

Stop before building the full browser. First confirm whether the original export itself has that gap. Do not invent or rename missing numbered files. The chunk numbers are not dates, but a missing chunk can mean missing conversations.

### Why the manifest matters

**The manifest turns an invisible preparation step into an auditable one.** You can see exactly which conversation JSON files were copied and where each came from inside the original export. That makes it easier to explain, reproduce, and troubleshoot the process later.

## 6 Manual fallback: collect and ZIP the JSON files yourself

Use this if you do not want to run the preparation helper.

### Manual collection works - but copy, do not move

The goal is to create a clean working folder. Never reorganize the only extracted copy of your export by moving files out of it.

1

#### Extract the original ZIP completely

Right-click the ChatGPT export ZIP and choose Extract All. Choose a location with plenty of free space and let extraction finish.

**Why this matters:** Windows search is more reliable on an extracted directory than inside a very large ZIP.

2

#### Search for conversations\*.json

Open the extracted export folder in File Explorer. Use the search box in the upper-right corner and enter conversations\*.json.

**Why this matters:** This narrows the export to the conversation record files.

3

#### Inspect the names carefully

You should find either conversations.json or a numbered family such as conversations-000.json, conversations-001.json, and so on.

**Why this matters:** Do not collect every .json file in the export. Other JSON files serve different purposes.

4

#### Copy every relevant conversation file into a new empty folder

Create a folder named ChatGPT\_Conversation\_JSON and COPY the conversation JSON files into it.

**Why this matters:** A clean folder makes omissions and duplicates visible before you ZIP anything.

5

#### Sort by Name and verify the numbered sequence

If the export is numbered, switch File Explorer to Details view, sort by Name, and inspect the sequence from the lowest to highest number.

**Why this matters:** A missing chunk should be investigated before you continue.

6

#### Create the working ZIP

Select the collected conversation JSON files. On Windows 11 choose Compress to ZIP file. On older Windows versions use Send to > Compressed (zipped) folder. Name it ChatGPT\_Conversations\_Only.zip.

**Why this matters:** Now Stage 2 receives a compact, explicit conversation dataset.

### If both plain and numbered files appear

Prefer the numbered set for the working ZIP and investigate why both are present. Mixing both can duplicate conversations. The automated helper handles this conservatively for you.

## 7 Stage 2: build the local browser

Now the input is clean and easy to understand.

1

### Drag ChatGPT\_Conversations\_Only.zip onto run\_windows.bat

Drop the working ZIP on the Stage 2 launcher.

**Why this matters:** You are no longer asking the generator to reason about a huge mixed export - only the conversation corpus.

2

### Let the generator run

It parses conversation records, reconstructs the active transcript path, writes one HTML page per conversation, and creates a compact catalog.

**Why this matters:** One page per conversation scales far better than one gigantic HTML document.

3

### Wait for the browser to open

When complete, the launcher opens ChatGPT\_Archive\_Browser\index.html.

**Why this matters:** The browser is static HTML. No local server or cloud account is needed.

## The generated folder

```
ChatGPT_Archive_Browser\
index.html <-- DOUBLE-CLICK THIS LATER
catalog.json <-- machine-readable index
README.txt
conversations\
<one page per conversation>.html
```

### Important chronology rule

The browser does NOT use conversations-000.json, 001, 002, ... as a time sequence. Those are export chunks. It sorts conversations by each record's actual create\_time timestamp.

## Optional command-line test

```
python chatgpt_archive_browser.py "ChatGPT_Conversations_Only.zip" -o browser-test --limit 100
```

*This is useful if you want to inspect 100 conversations before committing to the complete build. It is optional; the drag-and-drop workflow does not require it.*

## 8 Your first five minutes in the browser

Start by orienting yourself before doing deep analysis.

1

### Confirm the beginning and end

Use oldest-first and newest-first sorting. Make sure the dates roughly match the period you expect your account history to cover.

2

### Open the first conversation

Read it, then use Next to move chronologically. This is the simplest way to review a life or work history as a sequence rather than isolated chats.

3

### Search a distinctive project or phrase

Use a project name, device name, person, place, or repeated phrase. Open early and late matches to see development over time.

4

### Filter by year or broad topic

Use filters to reduce a large corpus to a manageable period or domain.

| Control       | Use it for                                 |
|---------------|--------------------------------------------|
| Search        | Titles plus indexed user text              |
| Oldest first  | Reconstructing beginnings and chronology   |
| Newest first  | Finding recent work quickly                |
| Title A-Z     | Scanning named projects or repeated titles |
| Longest first | Finding substantial conversations          |
| Year filter   | Reconstructing a specific period           |
| Topic filter  | Coarse exploration; labels are heuristic   |

### The browser is a map, not the territory

It is designed to help you find and read conversation text. The original export still holds attachments, images, and other assets that are intentionally not duplicated into the browser.

# 9 How this makes a huge history intelligible

The useful transformation is from storage format to human navigation format.

| Question you can now ask                       | Feature that helps                |
|------------------------------------------------|-----------------------------------|
| What was my very first conversation?           | True timestamp chronology         |
| Where did this project begin?                  | Search + oldest-first ordering    |
| How did an idea evolve?                        | Date order + Previous / Next      |
| What was I working on in a certain year?       | Year filter                       |
| Which conversations were substantial?          | Word and message counts           |
| Which conversations deserve deeper analysis?   | Opening prompt preview + metadata |
| How can another tool analyze selected history? | catalog.json                      |

## A productive review method

1. Read chronologically until a major thread becomes recognizable.
2. Write down a few distinctive terms for that thread.
3. Return to the index and search those terms across the whole corpus.
4. Compare the earliest and latest matching conversations.
5. Use year filters to locate periods of concentrated work.
6. Only then select a smaller set of conversations for AI-assisted synthesis or publication.

### Structure before synthesis

A giant ZIP contains evidence but very little orientation. Chronology, search, previews, and a catalog let a human identify the meaningful subsets before asking another system to summarize or interpret them.

# 10 Privacy and backup rules

The public software is safe to share. Your private output usually is not.

## Never publish the conversation-only ZIP by accident

ChatGPT\_Conversations\_Only.zip contains your conversation text. The generated HTML pages also contain readable copies. Do not put either in a public GitHub repository or website unless you deliberately sanitize them.

- Keep the original ChatGPT export ZIP as an immutable backup.
- Keep the working ZIP and generated browser on a local or encrypted drive when appropriate.
- Publish the generator, launcher scripts, and documentation - not your personal archive.
- If you delete the generated browser, you can rebuild it later from the preserved export.
- Use the original export when you need files or media that the text browser intentionally leaves behind.

## Local-first behavior

- The preparation helper and browser generator use local Python file I/O.
- The working ZIP is created on your computer.
- The generated archive opens with local file:// URLs.
- No database server, package manager, browser extension, or cloud upload is required.

# 11 Troubleshooting for first-time users

Most problems have a simple checkpoint.

| What you see                               | What to do                                                                                                             |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Python 3 was not found                     | Install Python 3, enable Add Python to PATH if offered, close the window, and retry.                                   |
| No conversation JSON files were found      | Confirm you dragged the original ChatGPT export ZIP, not another ZIP from inside it.                                   |
| Working ZIP is far smaller than the export | Expected. The helper intentionally leaves out media and attachments.                                                   |
| Gap warning in numbered files              | Stop and inspect the source export. Do not create fake files or rename chunks to hide the gap.                         |
| Stage 1 succeeds but Stage 2 fails         | Open the working ZIP and confirm it contains conversation JSON plus the manifest; preserve both ZIPs while diagnosing. |
| Browser opens but old images are missing   | Expected. The tool is a conversation-text browser; consult the original export for assets.                             |
| Topic label is wrong                       | Topic labels are lightweight navigation aids, not authoritative semantic analysis.                                     |
| The browser folder was deleted             | Rebuild it from ChatGPT_Conversations_Only.zip, or regenerate that ZIP from the original export.                       |

## When in doubt, return to the checkpoints

You should always be able to answer: (1) Is the original ZIP safe? (2) Did Stage 1 create and verify the conversation-only ZIP? (3) Did Stage 2 create index.html? Those three checkpoints isolate almost every setup problem.

## 12 Quick reference

Preserve -> Collect -> Verify -> Build -> Browse.

### Windows drag-and-drop path

| Stage | Action                                                   | Result                             |
|-------|----------------------------------------------------------|------------------------------------|
| 1     | Drag ORIGINAL export ZIP onto prepare_windows.bat        | ChatGPT_Conversations_Only.zip     |
| 2     | Drag ChatGPT_Conversations_Only.zip onto run_windows.bat | ChatGPT_Archive_Browser\index.html |

### Command-line equivalents

```
python prepare_conversations_zip.py "chatgpt-export.zip" -o ChatGPT_Conversations_Only.zip

python chatgpt_archive_browser.py "ChatGPT_Conversations_Only.zip" -o
ChatGPT_Archive_Browser
```

### Public project

[ChatGPT Archive Browser on GitHub](#)

The project contains the preparation helper, browser generator, Windows launchers, README, license, and this guide.

### Official OpenAI export reference

[OpenAI Help Center - Transfer exported conversations between ChatGPT accounts](#)

OpenAI documents that larger exports may contain numbered conversation JSON files rather than one conversations.json file, and that uploading those files does not recreate the old sidebar. Accessed September 17, 2026.

#### Success is simple to recognize

You have succeeded when the original export is still safe, the conversation-only ZIP has been verified, and index.html lets you search and read the history chronologically.